GOSIM®
GLOBAL OPEN-SOURCE INNOVATION MEETUP

GOSIM Shenzhen 2026

智能体操作系统与应用

Agentic OS & App



GOSIM®
GLOBAL OPEN-SOURCE INNOVATION MEETUP

GOSIM Shenzhen 2026

Topic

Harness智能模型发布： JIT-Agent

Introducing Harness Intelligence Model: JIT-Agent



张桂彬 Guibin Zhang

新加坡国立大学博士生

PhD Student, National University of Singapore

GOSIM

GOSIM Shenzhen 2026

JIT-Agent-27B

Scaling Harness Intelligence via Just-in-Time Harness Evolution

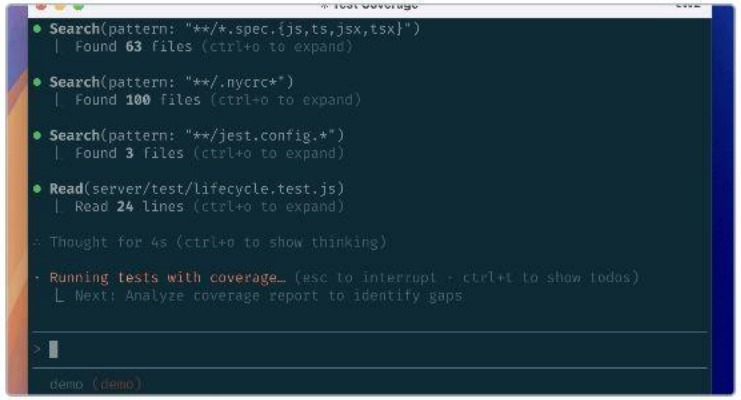
LV-NUS Lab · EverMind AI



Production agent harnesses

Mature interfaces already organize how models work.

Claude Code



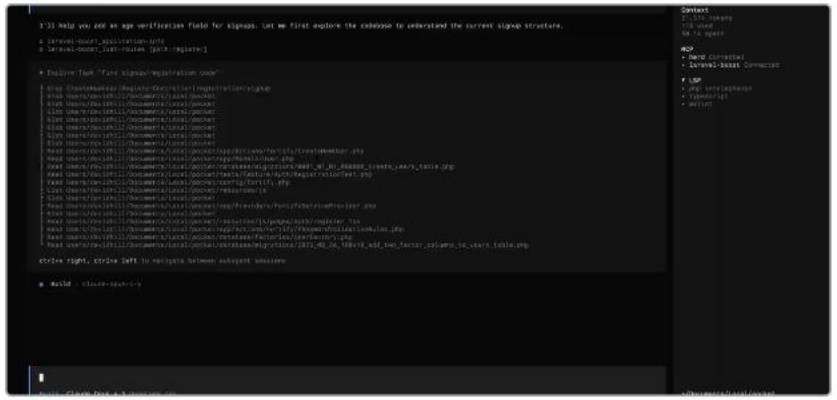
Repository tools · terminal execution

Codex



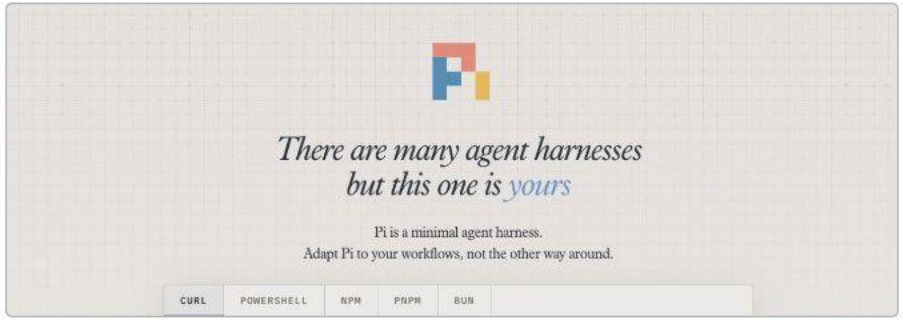
Plans · worktrees · skills · artifacts

OpenCode



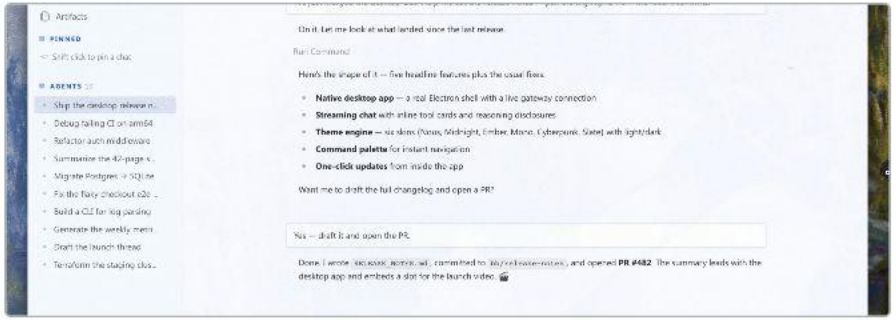
Multi-provider coding runtime

Pi



Minimal core · extensible workflows

Hermes



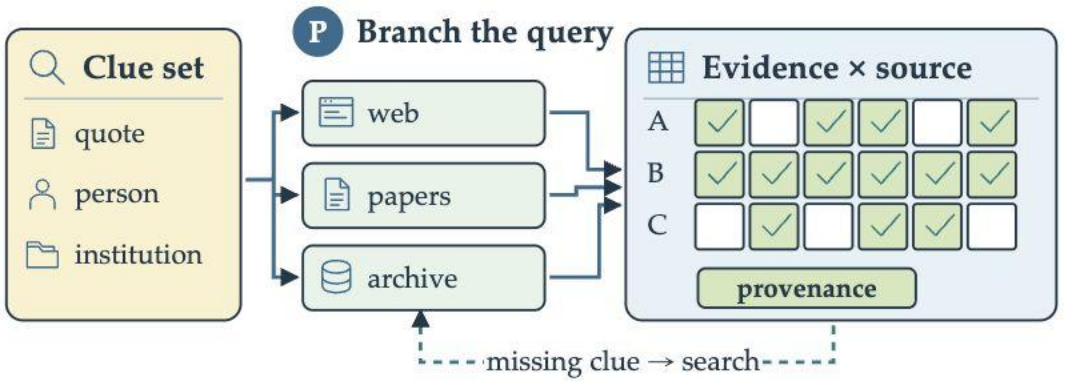
Persistent memory · scheduled work

The harness
decides what is
remembered,
what can be called,
and how work proceeds.

The model and its operating environment form the agent.

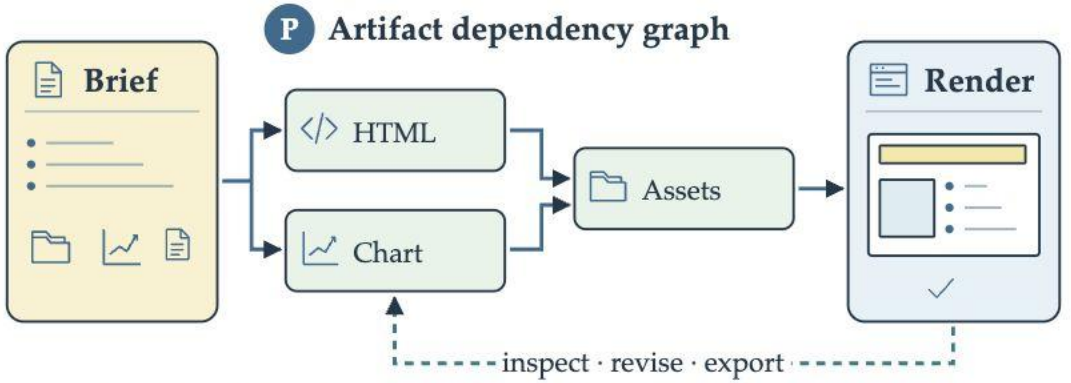
Different work calls for different harnesses

Deep research



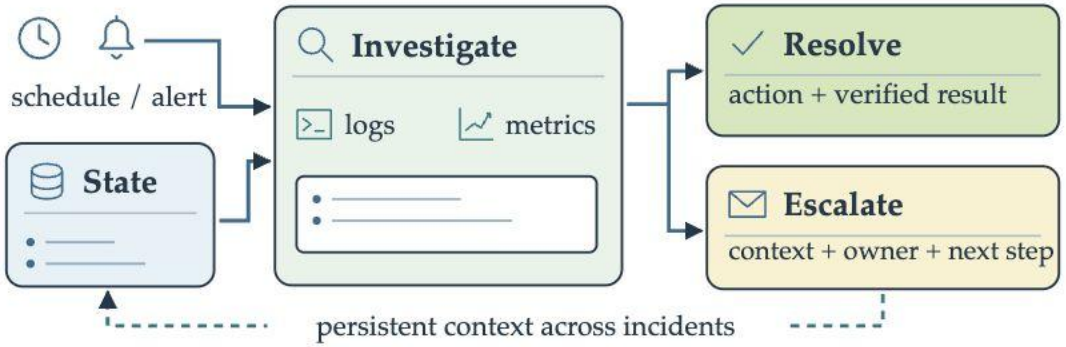
Six clues → a sourced identity; track missing evidence

Artifact production



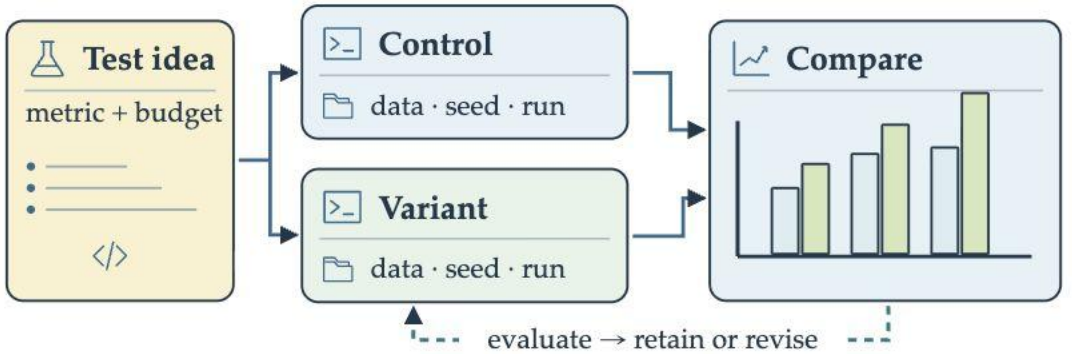
Codex: artifact tools + render–inspect–revise workflows

Daily on-call



Hermes: persistent memory + scheduled tasks + messaging

Automated research



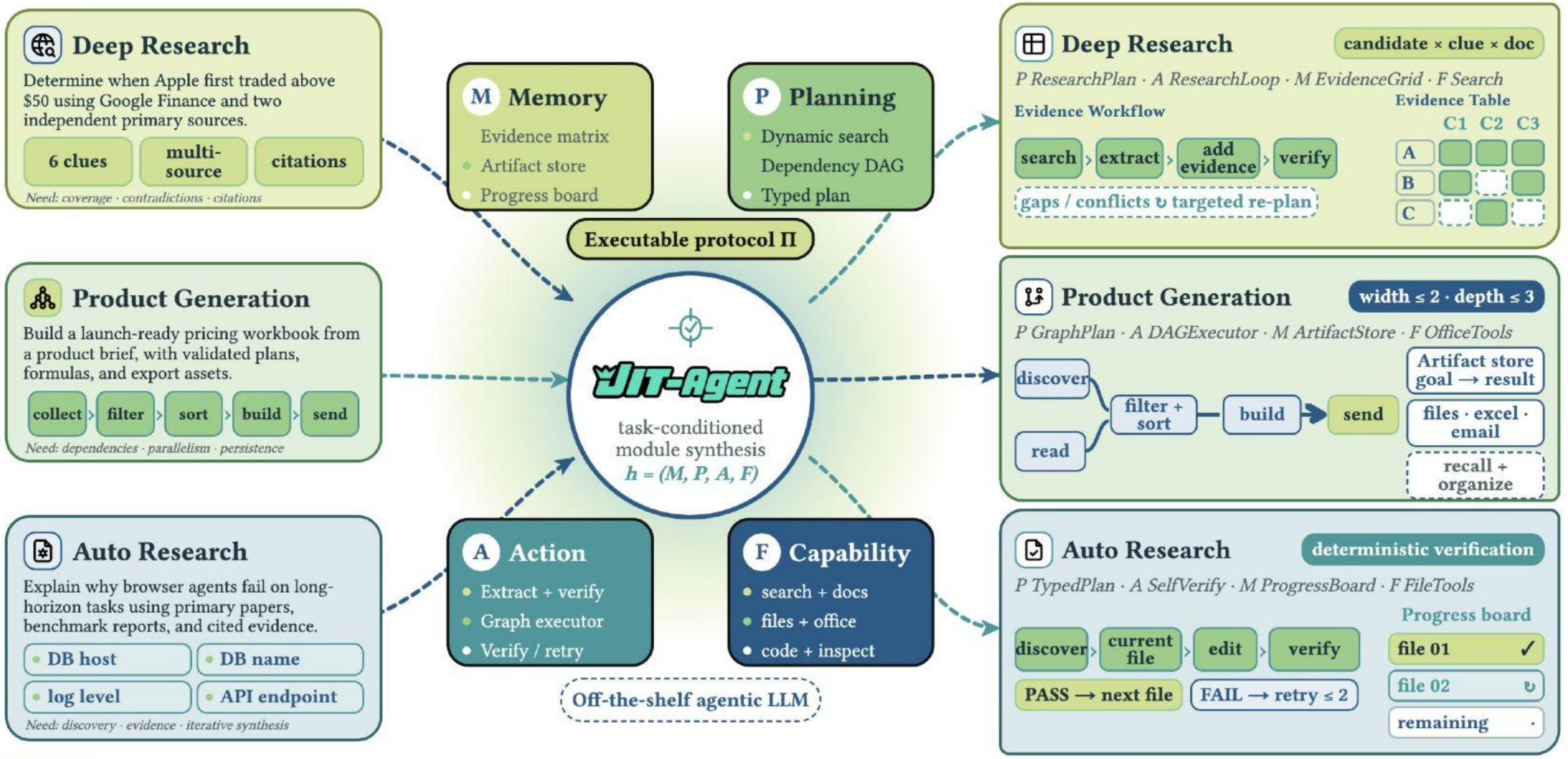
Code change → controlled comparison → measured result

Task structure changes the memory, control flow, and tool policy that are useful.

Can we build a custom "Claude Code" for every user request?

Just-in-time harness generation

Generate a task-specific harness. Let an off-the-shelf agentic LLM execute it.



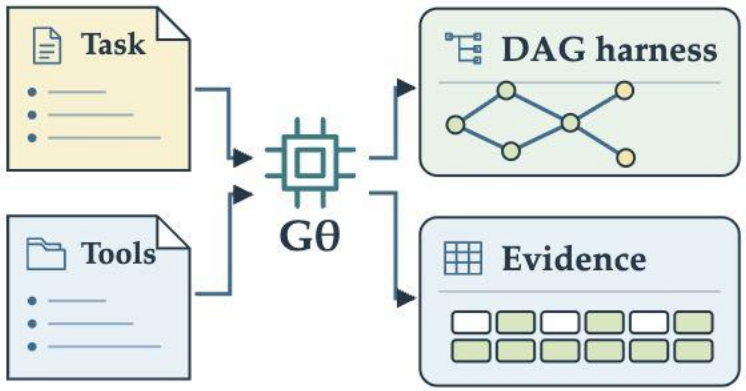
Harness intelligence

Harness Intelligence

The capacity to construct and refine the operational scaffold through which a model acts.

It matches the scaffold to the task and backbone, produces executable behavior and recovery, and turns execution feedback into stronger future harnesses.

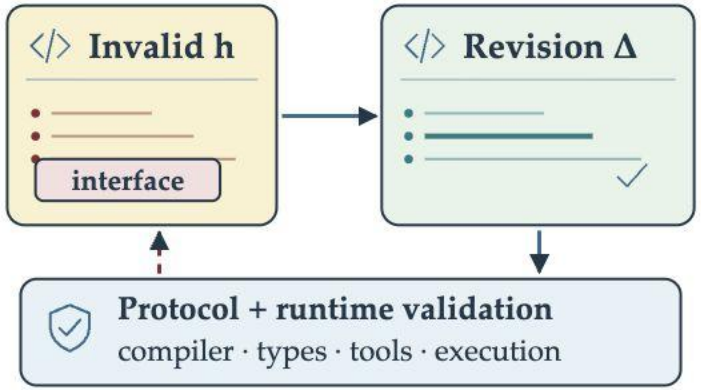
1 Task-conditioned construction



Adaptivity

Different tasks → different programs

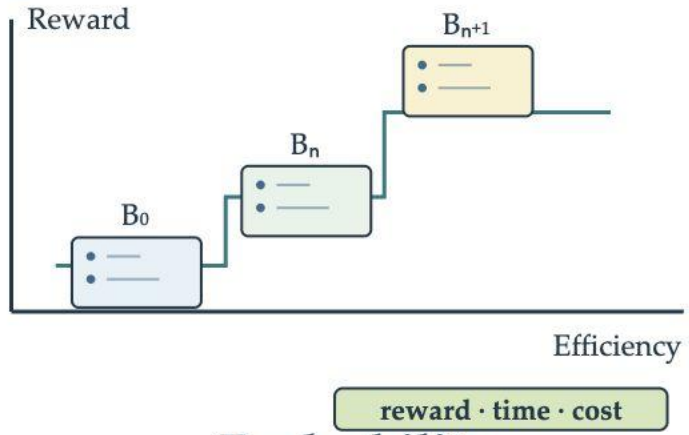
2 Executable recovery



Reliability

Diagnostics → patch → revalidate

3 Improvement from experience



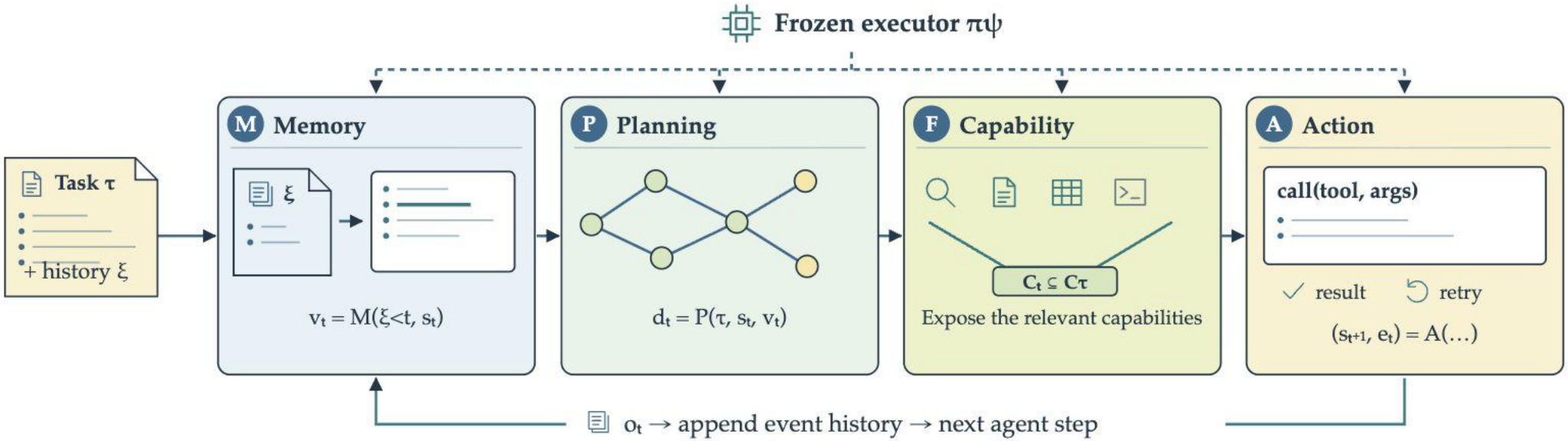
Evolvability

Archive stronger executable designs

A model learns both **how to operate** and **how to improve the way it operates**.

Codex and Claude Code:
500K+ lines of code each.
Can a single task query
generate a harness?

Four modules, one executable protocol



Harness	M · Memory	P · Planning	F · Capability	A · Action
ReAct	Full history	No explicit planner	Full registry	Serial reason–act loop
GAM	Retrieved memory	DAG plan	Full registry	ReAct execution
ROMA	Isolated subproblem state	Atomizer + DAG	Full registry	Recursive execution
Turnstile (generated)	Travel evidence buckets	Typed travel checklist	Gate final_answer	Collect until complete

Modularization makes just-in-time harness generation possible.
Fixed interfaces; task-specific implementations of M, P, F, and A.

HarnessFactory

Seed harness	Memory	Planning	Action	Capability
ReAct	FullHistory	None	ReAct	Full registry
Plan-and-Execute	FullHistory	Linear roadmap	ReAct	Full registry
ReSum	ReSum memory	None	ReAct	Full registry
Flash-Searcher	FullHistory	DAG	ReAct	Full registry
GAM	GAM retrieval	DAG	ReAct	Full registry
MemoBrain	Reasoning graph	None	Marker-guided	Full registry
AggAgent	Isolated rollouts	None	Multi-rollout	Full registry
OAgent	Coordinator history	None	Ensemble voting	Full registry
AgentFold	AgentFold memory	DAG	ReActFold	Full registry
HiAgent	Hierarchical memory	None	ReAct	Full registry
DeepAgent	Three-tier memory	None	Marker-guided	Tool search
ROMA	Context isolation	Atomizer + DAG	Recursive	Full registry
AOrchestra	Context isolation	Atomizer + DAG	Recursive	Delegation

Shared execution semantics

```
h = (M, P, A, F)

view = M(history, state)
plan = P(task, view)
tools = F(registry, plan)
state, call = A(…)

observation = Exec(call)
```

Comparable modules

Executable seeds

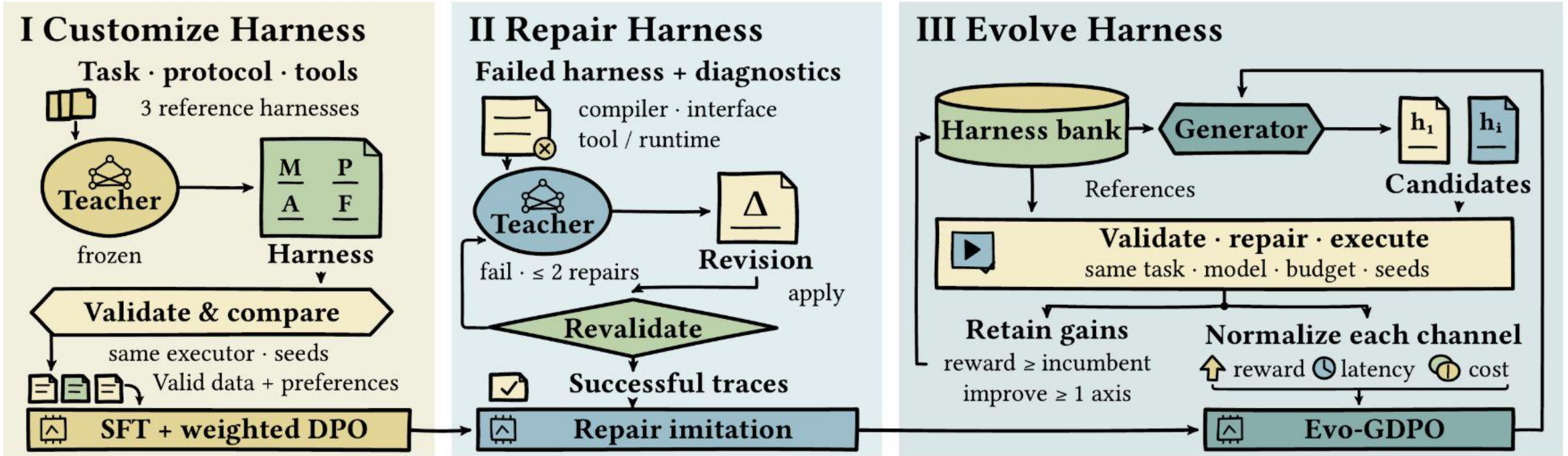
References for synthesis

Incumbents for evolution



HarnessFactory gives synthesis a **shared language of execution**, while preserving diverse designs.

Training JIT-Agent-27B

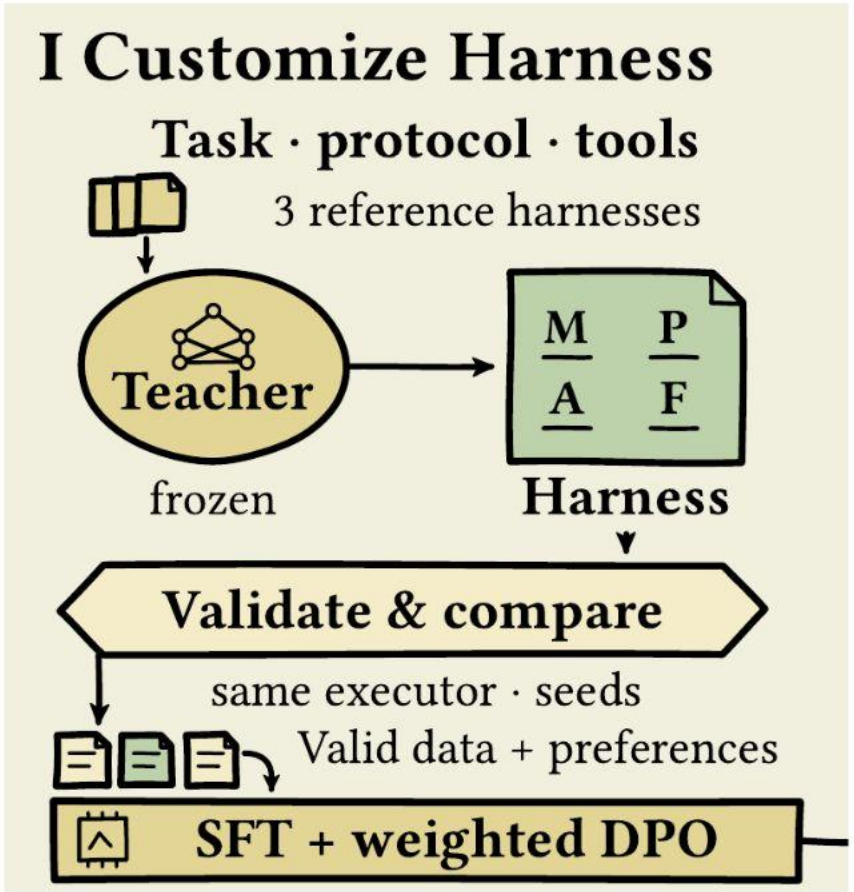


Source	BrowseComp-Plus	Monaco	WebWalkerQA	OdysseyBench-Neo	ClawGym
Tasks	300	1,315	200	302	500

Generator Qwen3.6-27B **Teacher** DeepSeek-V4-Pro-Preview **Executor** DeepSeek-V4-Flash-Preview

2,617 source tasks; training and test task instances are disjoint.

Stage I · Customize the harness



Task-conditioned teacher supervision

Task + protocol + tools + three seed references

```
teacher(task, Π, tools, references)
    → { memory.py, planning.py,
        action.py, capability.py }

retain only executable demonstrations
```

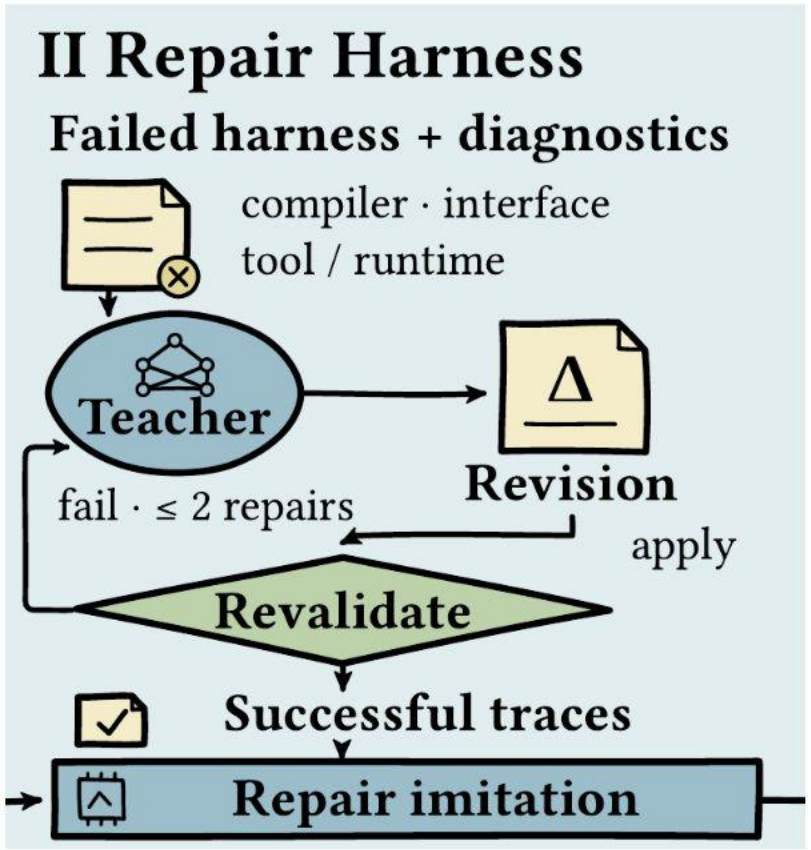
Preference learning under the same executor

Pair rule	Preferred h^+	Rejected h^-
Task reward	Strictly higher	Lower
Latency + cost	Neither worsens	Dominated
Efficiency	At least one strict gain	—

SFT learns valid structure. DPO rewards useful, efficient implementations.

Preference weights combine reward gain, latency reduction, and monetary savings.

Stage II · Repair the harness



Learn from failed execution

```
Illustrative interface failure

Action.run() returned a raw string
Expected: (next_state, executable_call)
```

```
Teacher revision Δ

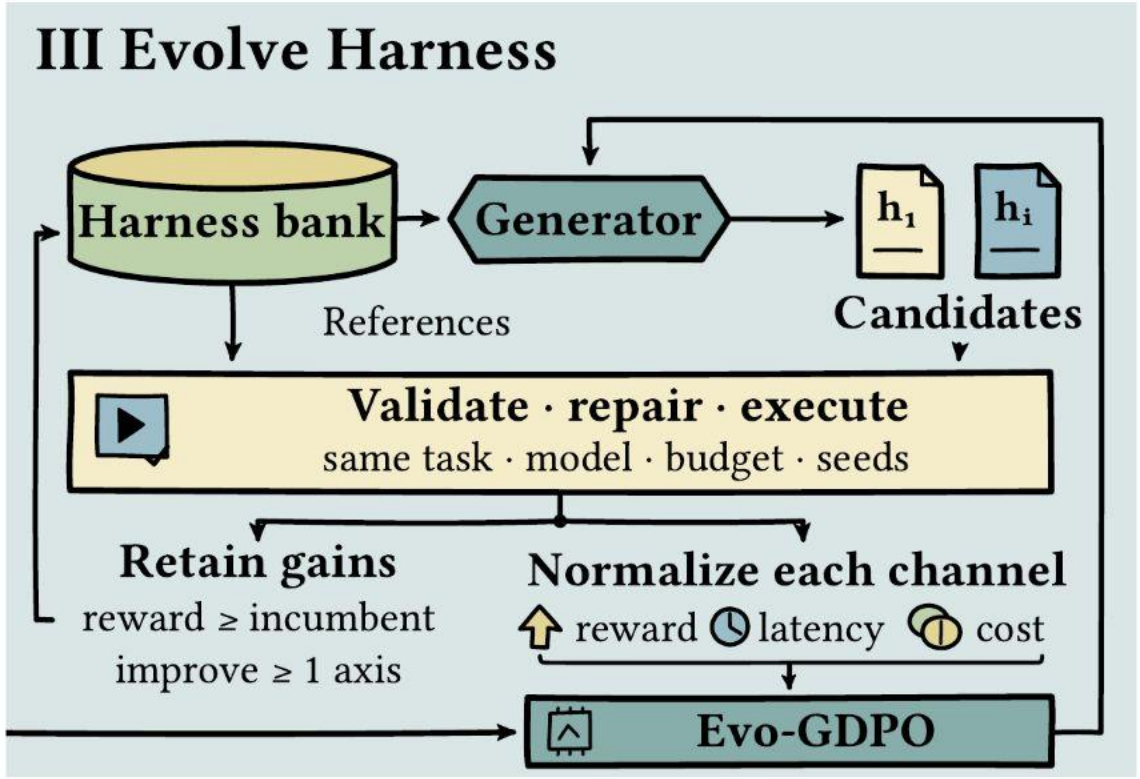
return next_state, ToolCall(name, args)

apply patch → validate → execute
```

Checkpoint	Valid@0	Valid@1
Before repair training	88.6%	90.2%
After Stage II	95.2%	**98.3%**

✦ One repair recovers about 65% of initially invalid harnesses after Stage II.

Stage III · Evolve the harness



During training: update generator weights θ and harness bank B_n.

Compare against the archive incumbent

$$R_r = r + \lambda_{evo}[r - b_r]_+$$
$$R_l = \mathbb{1}[r \geq b_r]\,[b_l - \ell]_+$$
$$R_c = \mathbb{1}[r \geq b_r]\,[b_c - \kappa]_+$$

Normalize each channel within the group

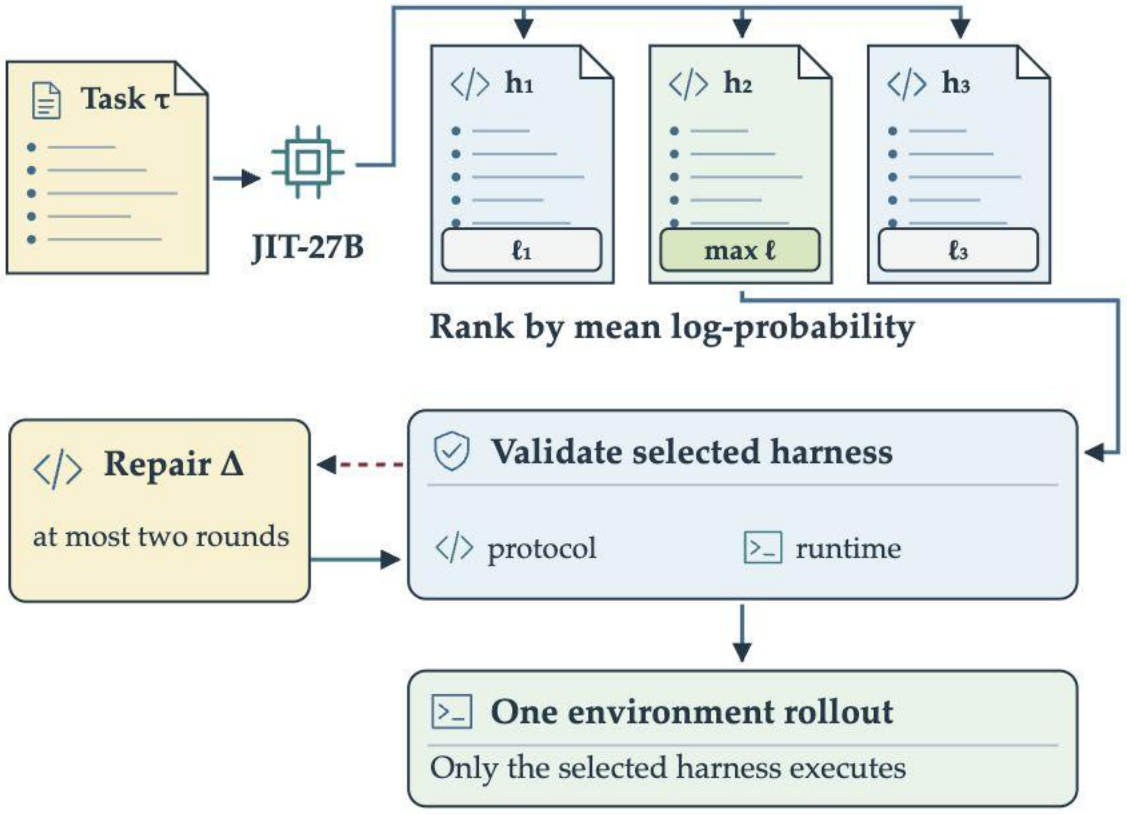
$$A = 0.8\,A^{reward} + 0.1\,A^{latency} + 0.1\,A^{cost}$$

Optimization	Configuration
Candidates / task	G = 4
Tasks / update	16 tasks · 64 candidates
Policy update	Clipped PPO + KL
Archive admission	Pareto dominance

✦ Reward improvement comes first. Efficiency gains count when task reward is preserved.

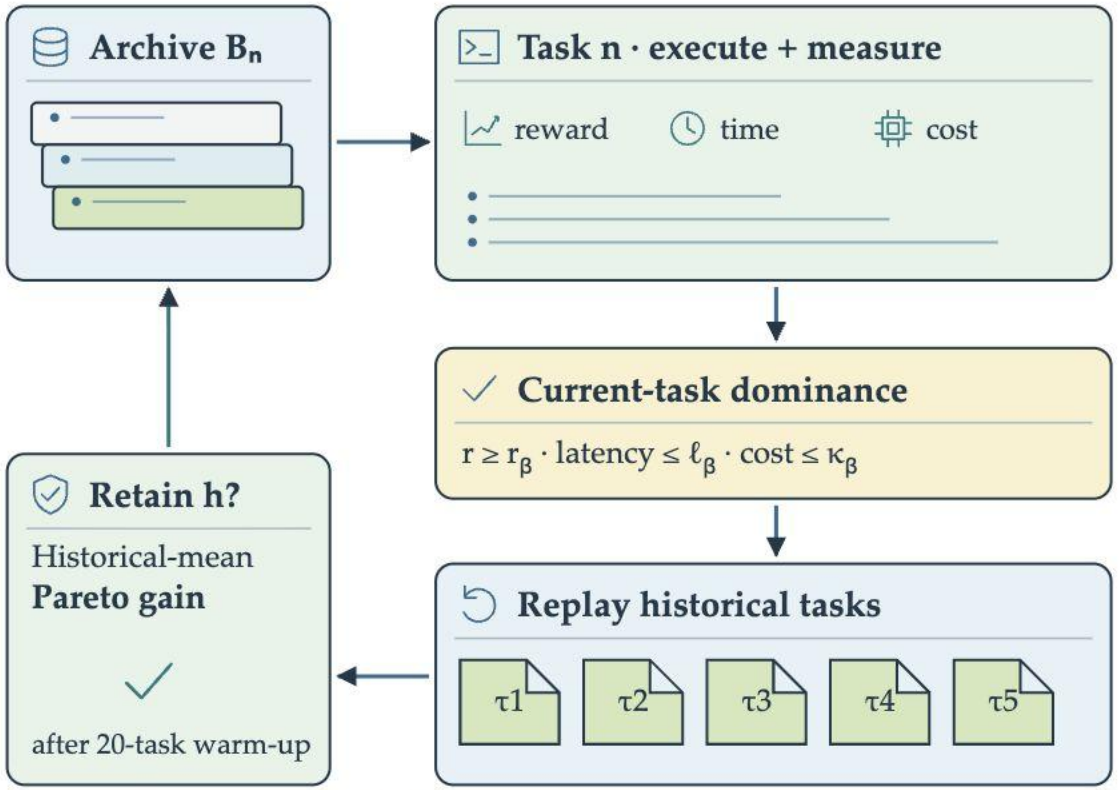
Static and streaming inference

Within a task



Candidate diversity without extra scored rollouts

Across tasks



θ is frozen; the archive changes

Streaming experience is stored in the harness archive, with the generator frozen.

Can JIT-Agent unlock a leap in performance for existing models?

Task-adaptive harnesses strengthen fixed backbones

Model	BC+	DSQA	xBench	AgentIF	Pinch	Shop	Travel	Office	Odyssey
Qwen3.7-Plus	70.5	78.0	75.0	59.9	80.5	75.2	52.1	56.6	67.7
GLM-5.2	72.0	89.2	76.0	63.0	87.0	78.2	62.8	63.0	75.3
DeepSeek-V4-Flash	68.1	76.2	70.0	58.4	81.7	59.1	54.8	61.0	71.0
DeepSeek-V4-Pro	71.4	72.4	79.0	56.5	61.1	71.1	55.2	62.0	72.0
Kimi K2.7 Code	73.6	87.8	79.0	57.3	76.1	72.8	56.9	65.8	70.0
GPT-5.6-Sol	76.9	76.0	81.0	68.0	84.2	83.7	**84.9**	65.3	68.7
Gemini 3.1 Pro	67.5	75.8	83.0	60.1	81.0	77.4	70.7	60.6	74.0
Gemini 3.5 Flash	75.0	88.0	85.0	64.0	74.2	76.2	50.3	63.3	78.0
JIT + GLM-5.2	**78.0**	**93.9**	**88.0**	**69.9**	**93.3**	**83.4**	**83.0**	**68.4**	**78.7**
JIT + DeepSeek-Flash	**74.0**	**85.1**	**82.0**	**63.8**	**92.9**	**83.9**	**61.3**	**63.4**	**73.0**

DeepSeek-V4-Flash · Shopping

59.1 → 83.9 **+24.8**

GLM-5.2 · Travel

62.8 → 83.0 **+20.2**



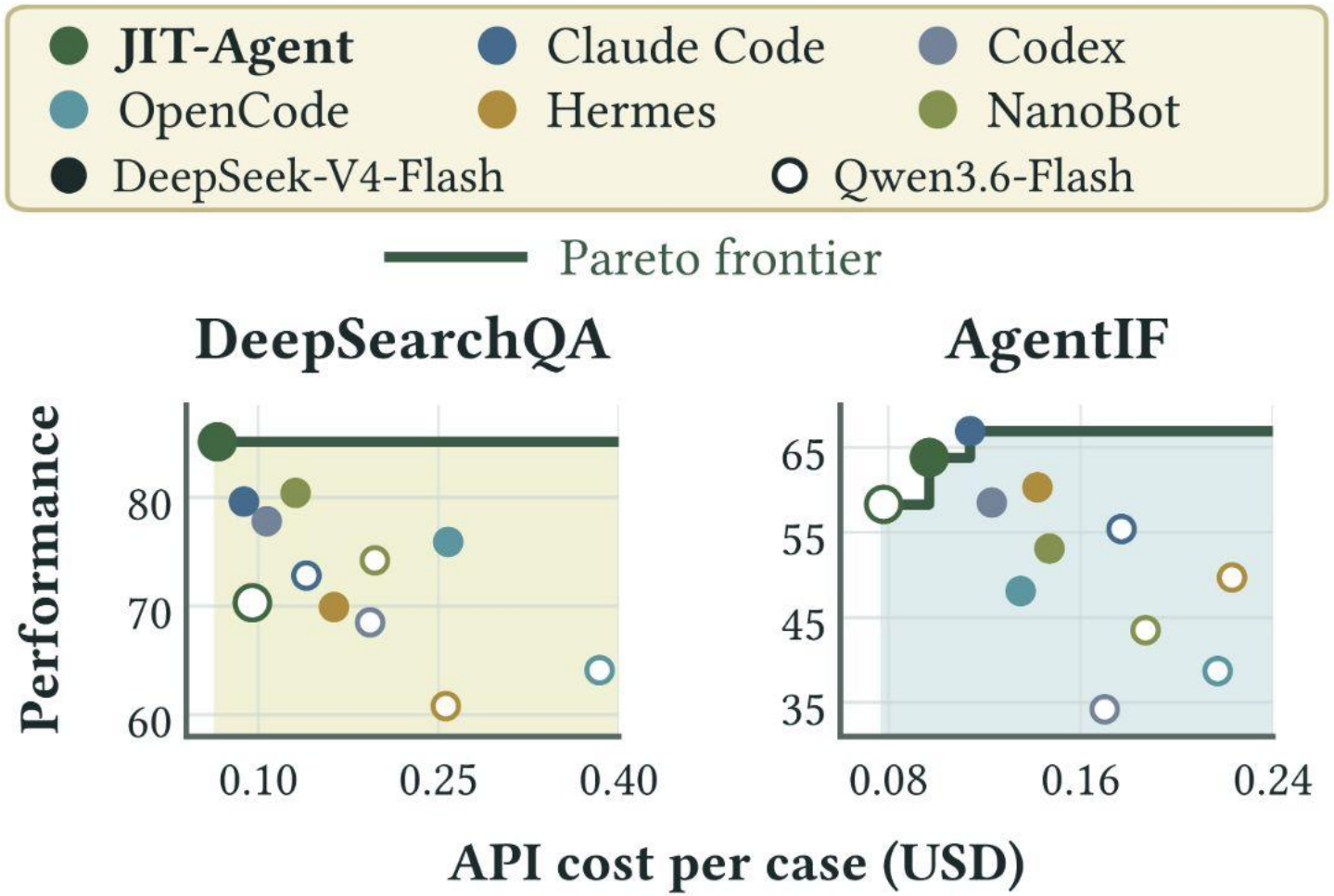
✦ Every matched backbone–benchmark pair improves: **18 / 18**. JIT leads **8 / 9** columns in this comparison.

Comparing JIT-Agent with production harnesses

		DeepSearchQA			xBench-DS			AgentIF-Oneday		
Executor	**Harness**	**Score**	**K tok**	**USD**	**Score**	**K tok**	**USD**	**Score**	**K tok**	**USD**
DeepSeek-V4-Flash	**Claude Code**	79.6	625	$0.088	75.0	559	$0.079	**66.9**	808	$0.114
	Codex	77.8	760	$0.107	70.0	680	$0.096	58.5	870	$0.123
	OpenCode	75.9	1,832	$0.258	65.0	1,157	$0.159	48.1	950	$0.135
	Hermes	69.9	1,157	$0.163	72.0	1,254	$0.177	60.3	1,000	$0.142
	NanoBot	80.4	924	$0.131	78.0	527	$0.075	53.1	1,034	$0.147
	JIT-Agent	**85.1**	**506**	**$0.066**	**82.0**	**212**	**$0.039**	**63.8**	**486**	**$0.097**
Qwen3.6-Flash	**Claude Code**	72.8	710	$0.140	58.0	650	$0.128	55.4	900	$0.177
	Codex	68.5	980	$0.193	52.0	874	$0.172	34.2	839	$0.170
	OpenCode	64.1	1,968	$0.384	44.0	1,300	$0.256	38.7	961	$0.217
	Hermes	60.8	1,319	$0.256	36.0	1,155	$0.227	49.7	1,184	$0.223
	NanoBot	**74.2**	892	$0.197	63.0	597	$0.119	43.5	950	$0.187
	JIT-Agent	**70.3**	**464**	**$0.095**	**70.0**	**300**	**$0.069**	**58.3**	**394**	**$0.078**

Lowest execution cost in all six settings. Highest score in four; generation is counted separately.

Performance, execution cost, and synthesis overhead



Generation overhead

Metric	DSQA	AgentIF
JIT generation time	**52 s**	**58 s**
Task execution time	552.8 s	583.6 s
Time ratio	9.4%	9.9%
JIT output tokens[1]	**18,107**	**24,526**
Task total tokens	506,880	486,776
Token ratio	3.6%	5.0%

[1] Three candidates + repair outputs.

Selected harness length:
5,424 / 7,790 tokens

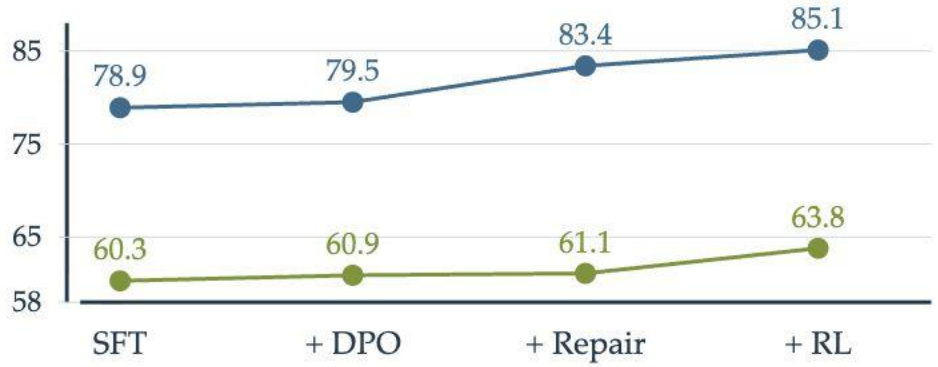
The generated program governs a much larger execution budget. Count synthesis separately.

What each training stage contributes

Module code size: tokens / lines

Checkpoint	DSQA	AgentIF	Valid@0	Valid@1	M tok	M line	P tok	P line	F tok	F line	A tok	A line	USD
Stage I · SFT	78.9	60.3	75.4	79.1	1290	170	771	102	501	70	2688	329	$0.090
SFT + DPO	79.5	60.9	88.6	90.2	1210	165	769	105	503	72	2703	335	$0.083
Stage II · Repair	83.4	61.1	95.2	98.3	1223	168	762	105	493	70	2719	330	$0.081
Stage III · Evo-GDPO	85.1	63.8	94.7	97.9	1382	180	758	103	515	76	2740	351	$0.077

Task performance DSQA AgentIF



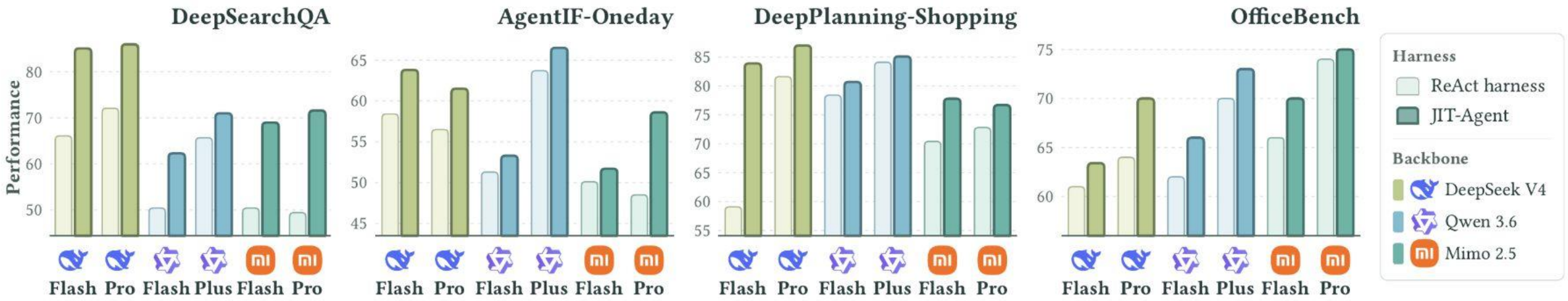
Execution validity Valid@0 Valid@1



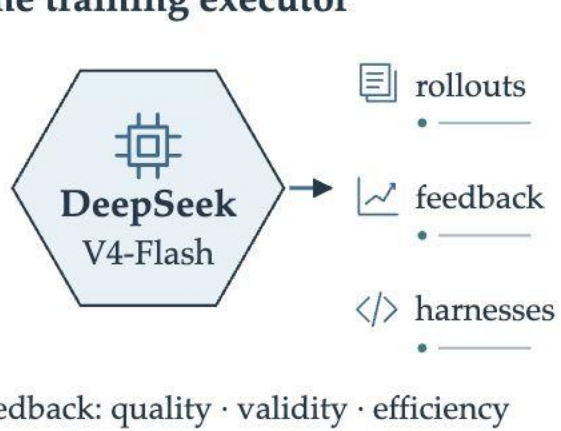


✦ Repair gives the strongest validity. Evo-GDPO gives the best task scores and lowest execution cost.

Transfer to executors unseen during training



One training executor



Feedback: quality · validity · efficiency

Generator θ stays fixed at test time

Different execution backbones

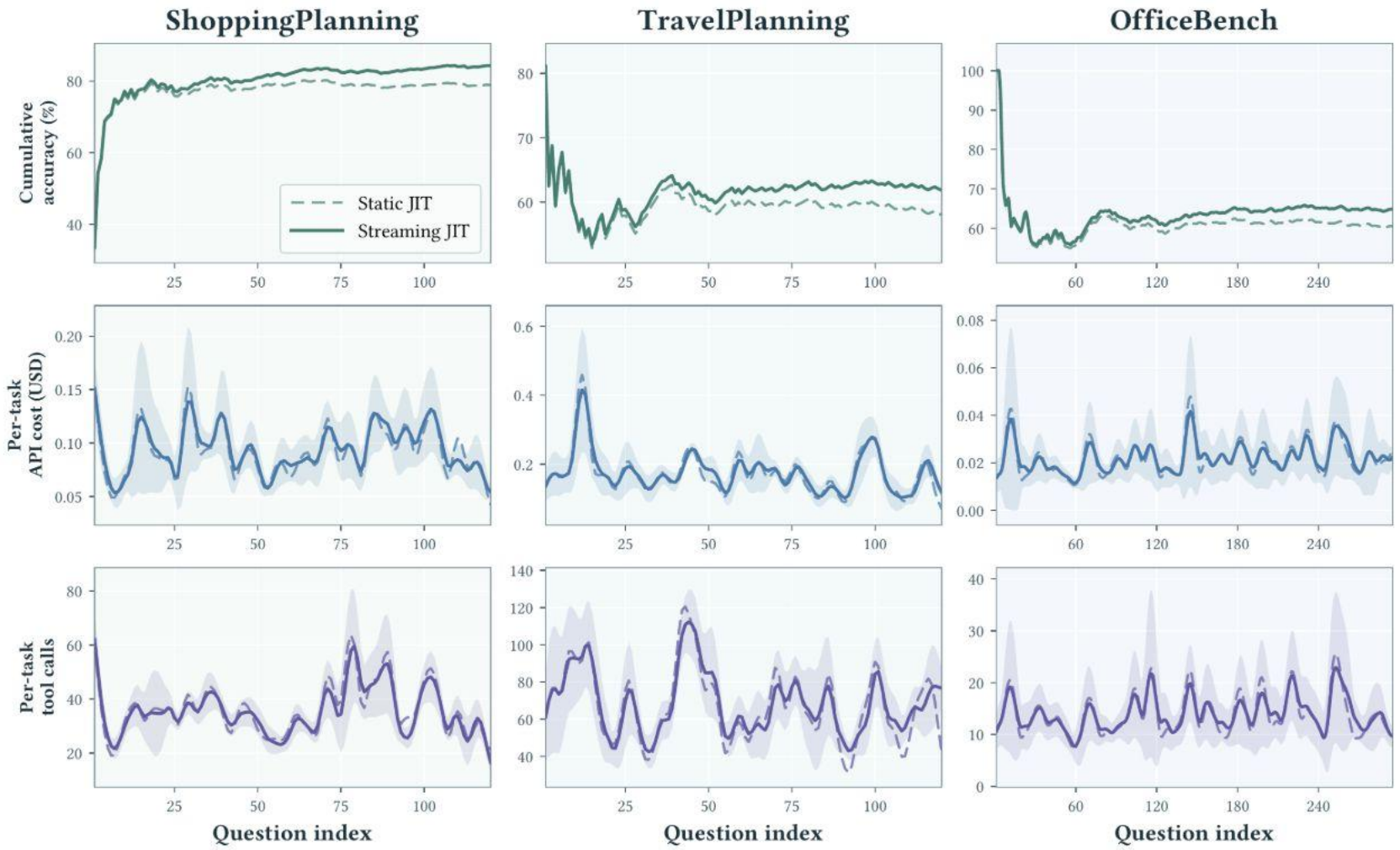
GLM-5.2
9 / 9 main benchmarks improve

Qwen3.6
Flash + Plus · +4.0 mean gain

Mimo-V2.5
Flash + Pro · +8.6 mean gain

✦ JIT wins all **24 matched comparisons** in the model-pair study, with a +7.6-point average gain.

Streaming harness evolution



Experience accumulates

Dashed: Static JIT
Solid: Streaming JIT

Higher final cumulative accuracy on all three streams.

Per-task cost and tool use remain task-dependent.

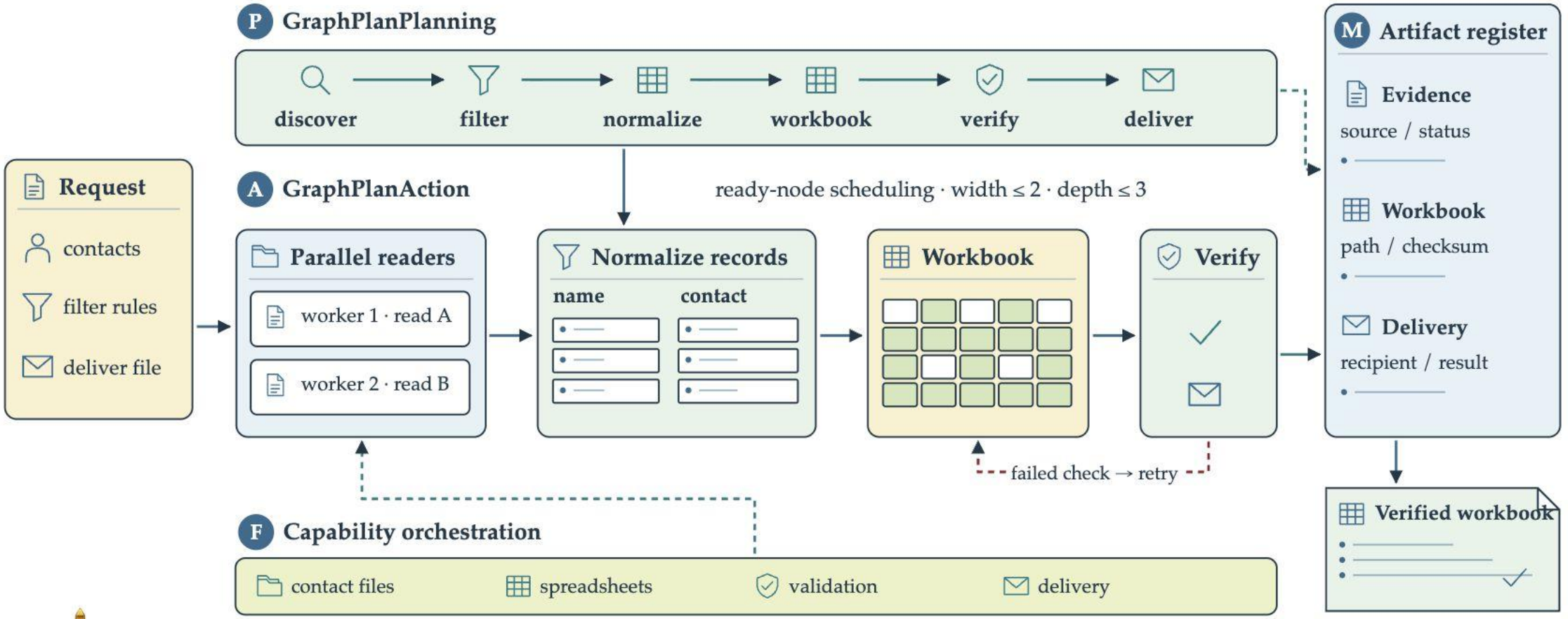
Admission rule

Current-task dominance
+ five-task replay
+ historical-mean dominance

The archive evolves across tasks. The generator parameters remain fixed.

Palimpsest · Artifact production as a dependency graph

TASK Collect all contact cards except Tom's, sort by name, build a workbook, then **email the verified file to Tom.**

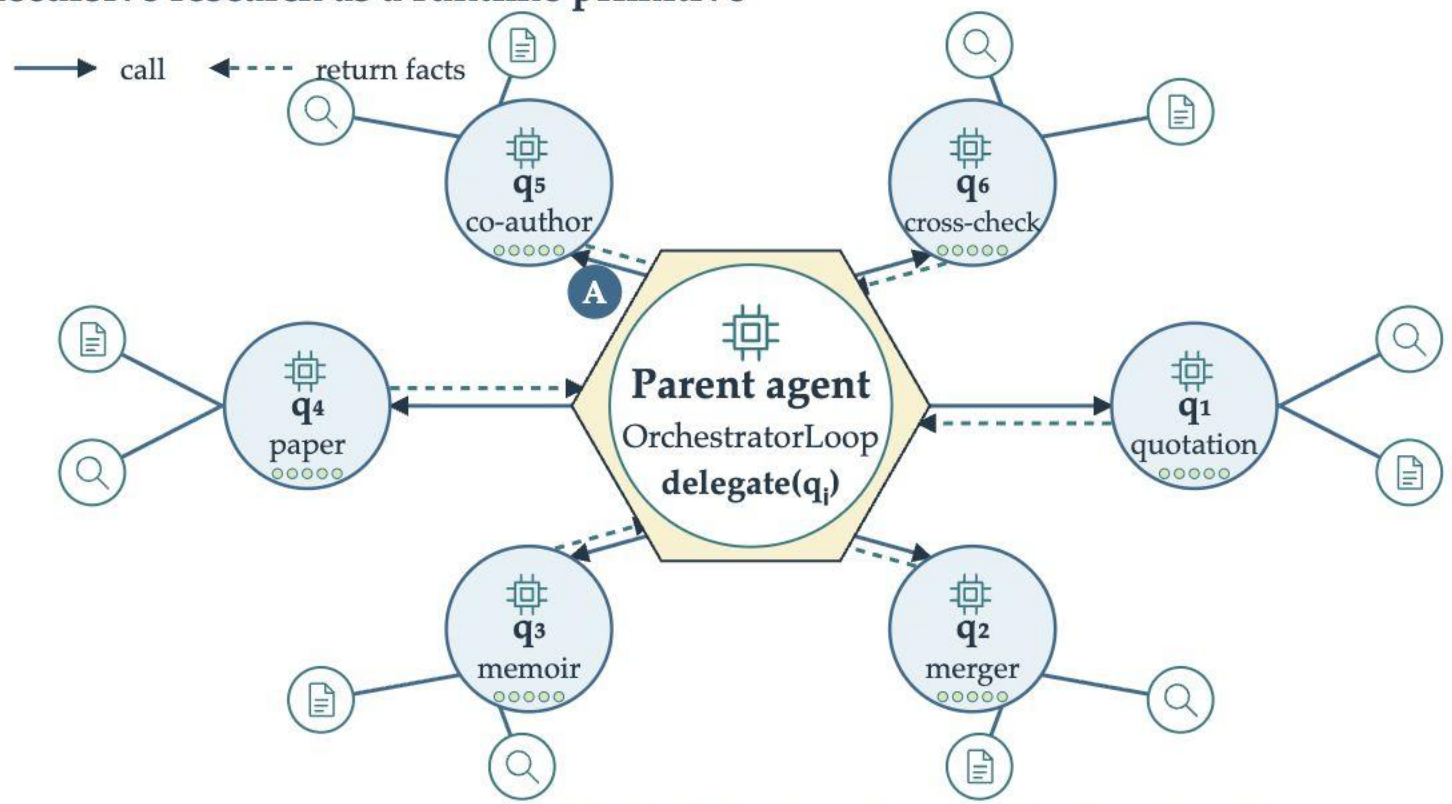


Artifact dependencies become executable control flow. Memory keeps each completed result.

Trapdoor · Research with bounded delegation

TASK Trace a 2010 paper through a 1901 John Muir quote, a 1974 university merger, and a 2023 memoir—then **identify the other co-author**.

Recursive research as a runtime primitive



Each child: private ξ_i · research-only tools · ≤ 5 steps

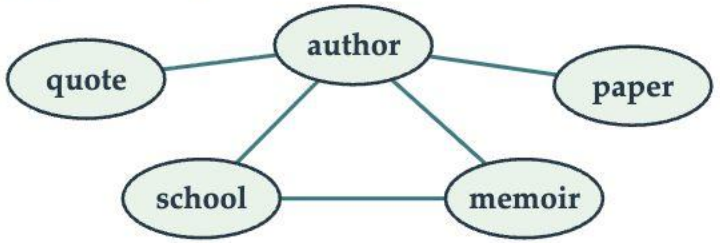
P DynamicDecomposer

Unresolved clue → subquestion

Revise the evidence path as facts arrive.

question → decompose → re-plan

M FactGraphMemory



{ key : value } + supporting source

Compact facts re-enter the parent as o_t.

F ResearchToolPolicy

search · read · delegate

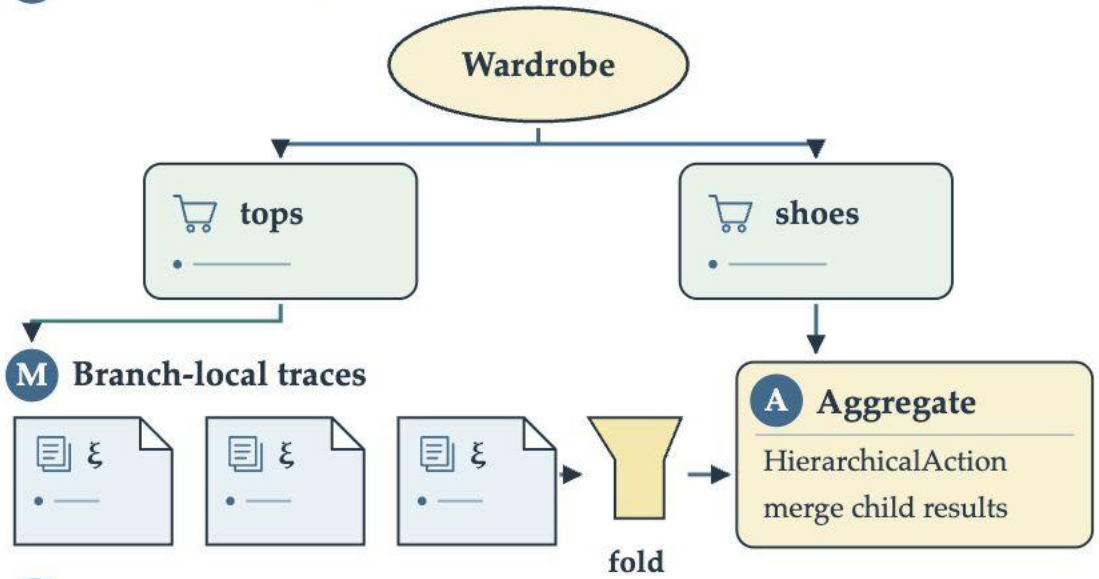
Illustrative branching structure · child answers return through the ordinary observation loop.

Memory can compress history or enforce coverage

Origami

TASK
Build a spring/autumn wardrobe: stock > 200,
delivery < 2 days, a size-40 companion item;
> 1,200 reviews, including ≥ 150 four-star reviews.

P ROMAPlanning · recursive decomposition



F Exposed capabilities

search · products · fold_thought

Full branch artifacts survive context folding.

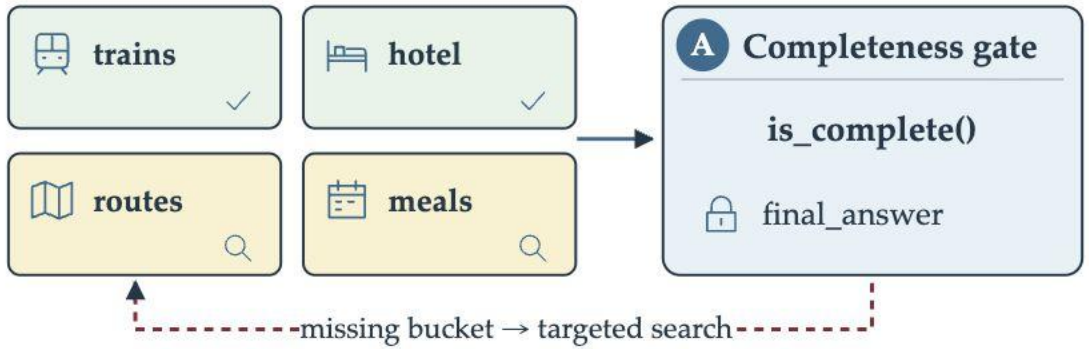
Turnstile

TASK
Plan Chongqing → Zhengzhou, 12–18 Nov 2025:
3 travelers, 2 rooms; train departure 07:00–11:00;
highest-rated Home Inn, top 3 attractions + daily routes.

P TravelPlanning · typed trip specification

dates · travelers · rooms · routes

M DataStoreMemory · coverage contract



F Exposed capabilities

missing facts · coverage check · gated answer

The itinerary is released only after coverage passes.

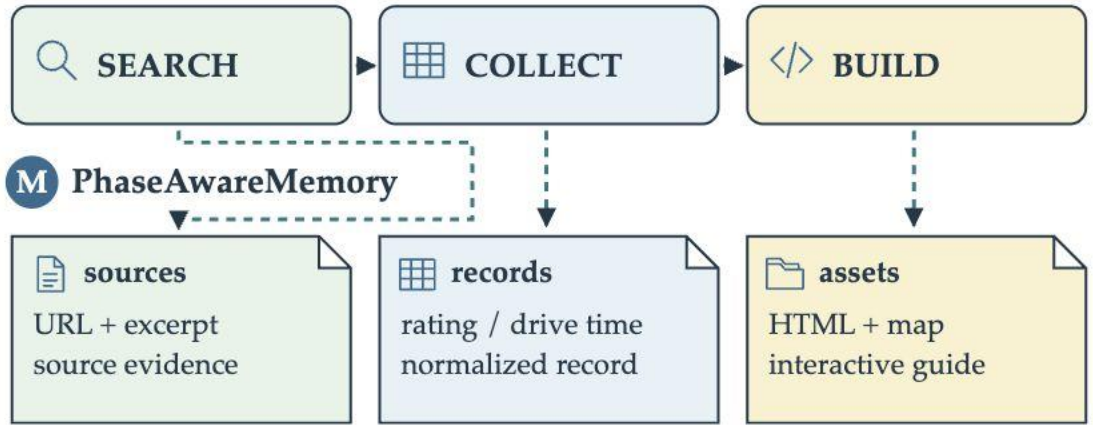
State can reshape the entire execution loop

Gearbox

TASK
Find good Chinese restaurants near 02155, all within a 45-minute drive, and publish an interactive one-page restaurant guide.

P Phase-conditioned intent

A PhaseAction · single writer of the phase register



F Exposed capabilities

web discovery · record inspection · construction

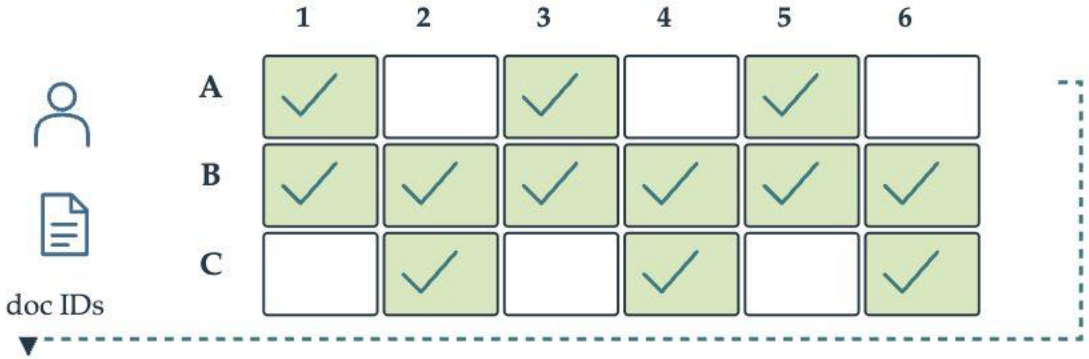
One phase change selects both schema and tools.

Pegboard

TASK
Which actor fits all six clues? Never married by 2023; 19th-c. US university; Guinness debut; hit film + sequel; 2019 reality-show guest; films in three languages.

P DynamicResearchPlanning · choose the next gap

M Candidate × clue × source



A Verify the winning candidate against original sources

F Exposed capabilities

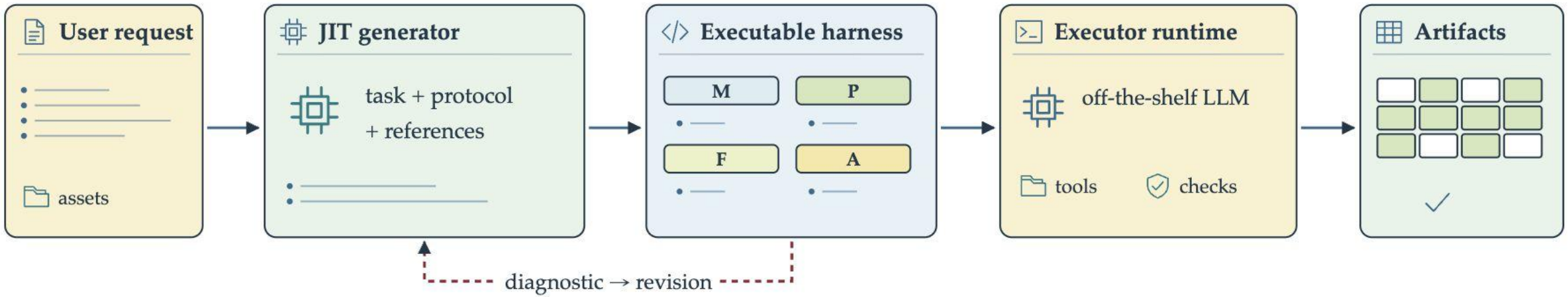
fill gaps · reopen sources · resolve conflicts

Evidence has coverage, provenance, and contradictions.

What does JIT-Agent mean for the next generation of foundation models?

A separate helper model is one deployment choice

Would users adopt a separate model just to generate their harness?

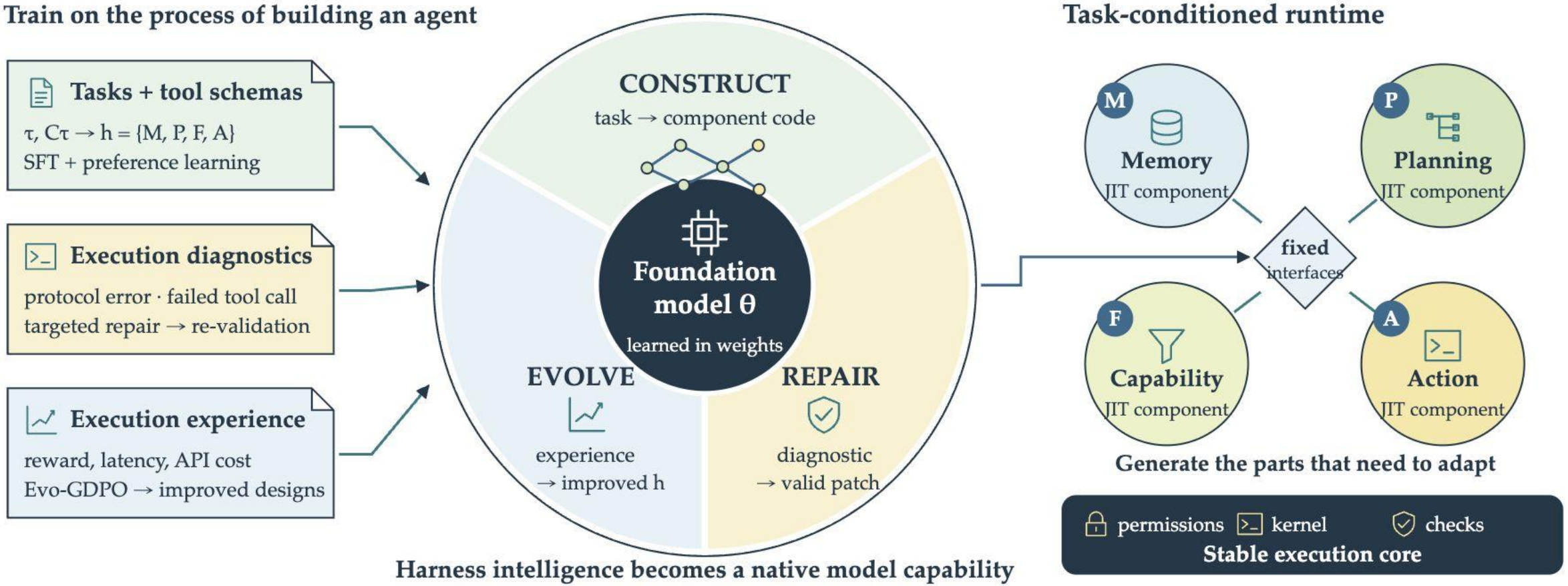


Evidence today	What it means for deployment
Final checkpoint: 94.7% Valid@0, 97.9% Valid@1	Keep validation and bounded recovery in the runtime
52–58 s generation in two measured settings	Amortize synthesis when tasks justify the overhead
Transfer from one training executor	Explore broader executor and task distributions

✦ Our strongest bet is the training recipe for future foundation models.

A recipe for the next foundation model

Next-generation foundation models should internalize harness intelligence.



Learn to construct task-specific components, repair from diagnostics, and evolve from experience.
JIT-Agent supplies a training recipe for these native capabilities.

GOSIM

GOSIM Shenzhen 2026

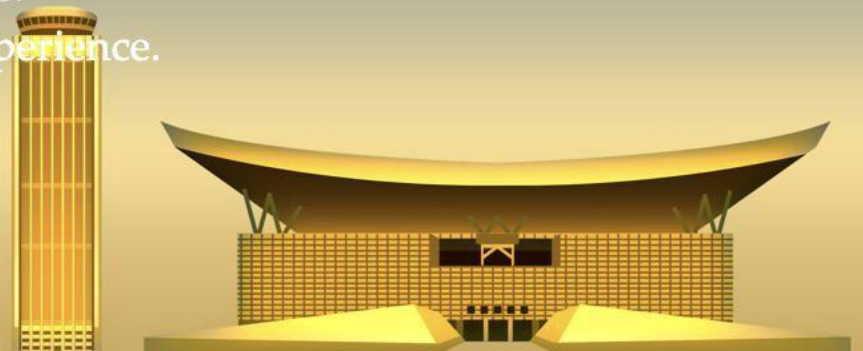
JIT-Agent-27B

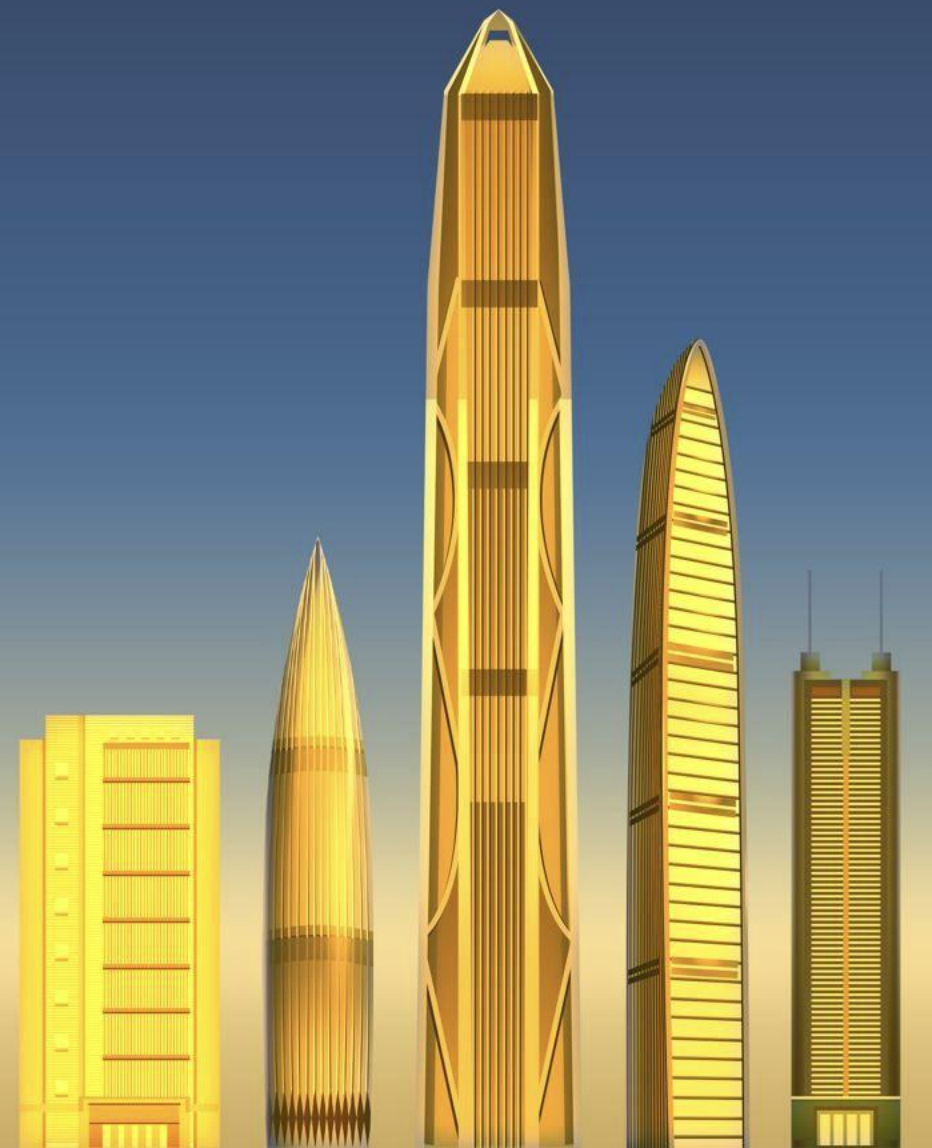
Scaling Harness Intelligence via Just-in-Time Harness Evolution

What should the next model learn about how it should operate?

Construct a task-specific scaffold.
Repair execution from diagnostics.
Improve future scaffolds from experience.







LV-NUS Lab · EverMind AI

Research prototype and training recipe

GOSIM®
GLOBAL OPEN-SOURCE INNOVATION MEETUP

GOSIM Shenzhen 2026

智能体操作系统与应用

Agentic OS & App

